ADVANCE NLP (AML23702)

LAB EXERCISE: 2

Student Name	Vijwal Sheshadri
USN	1GA23AI057
Experiment Title	Information Retrieval-based Document Search System

Aim / Objective

Purpose: States what the experiment sets out to build and why.

This experiment develops a simple information-retrieval system that accepts a user's search query and ranks a collection of text documents by relevance. Each document is represented as a TF-IDF vector. The query is converted using the same vocabulary, and cosine similarity is used to rank the documents. The system returns the most relevant document and a top-three ranking with similarity scores. It retrieves existing text and does not generate new content.

Expected Outcomes Addressed

Purpose: Maps the work to the expected outcomes of the experiment.

- **Outcome 1: Accept a search query** — A query box accepts a natural-language phrase.
- **Outcome 2: Represent documents** — TfidfVectorizer converts document text into numerical features.
- **Outcome 3: Rank relevant documents** — Cosine similarity compares the query vector with document vectors.
- **Outcome 4: Display relevance scores** — The interface displays a ranked list with scores.
- **Outcome 5: Test different queries** — Multiple queries are tested against the sample corpus.

Tools & Technologies Used

Purpose: Gives a technical inventory of the system.

Category	Tool / Library	Version	Purpose
Language	Python	3.x	Core retrieval logic
Vectorisation	scikit-learn TfidfVectorizer	1.x	Term weighting and text vectorisation
Similarity	scikit-learn cosine_similarity	1.x	Query-to-document relevance scoring
Interface	HTML, CSS, JavaScript	—	Simple search interface
Visualisation	Pillow / Matplotlib	—	Output screenshot and score display

Dataset / Corpus Description

Purpose: Establishes what text data is used for testing.

A small demonstration corpus of four documents is used. The documents cover natural language processing, TF-IDF, named entity recognition, and information retrieval. This compact dataset makes the ranking easy to inspect and suitable for a lab demonstration. The corpus can be expanded by adding more documents.

Sample Text Records:

- Document 1: Natural Language Processing enables computers to process human language.
- Document 2: TF-IDF represents documents using term importance weights.
- Document 3: Named Entity Recognition identifies names, places and organisations.
- Document 4: Information retrieval finds relevant documents for a user query.

System Architecture / Pipeline Design

Purpose: Shows the end-to-end workflow.

```
Document collection
        |
        v
Text cleaning and tokenisation
        |
        v
TfidfVectorizer.fit_transform(documents)
        |
User query --> vectorizer.transform(query)
        |
        v
Cosine similarity scores
        |
        v
Sort scores descending --> Top-ranked documents
```

- **Module 1 (Input):** Accepts the document collection and search query.
- **Module 2 (Preprocessing):** Converts text to a consistent representation.
- **Module 3 (Indexing):** Builds TF-IDF vectors for all documents.
- **Module 4 (Scoring):** Computes cosine similarity between query and each document.
- **Module 5 (Results):** Displays the highest-scoring documents in descending order.

Backend Implementation

Purpose: Explains the core retrieval logic.

6.1 Approach / Algorithm

- **TF-IDF:** Gives higher weight to terms that are frequent in one document but less common across the full collection.
- **Cosine similarity:** Measures the angle between query and document vectors. Higher scores indicate stronger term overlap.
- **Ranking:** Sorts documents by score and returns the top three results.
- **Empty-query handling:** The program checks that a query is provided before searching.

6.2 Key Code Snippet

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

documents = [
    "Natural Language Processing enables computers to process human language.",
    "TF-IDF represents documents using term importance weights.",
    "Named Entity Recognition identifies names, places and organisations.",
    "Information retrieval finds relevant documents for a user query."
]

vectorizer = TfidfVectorizer(stop_words="english")
document_vectors = vectorizer.fit_transform(documents)
```

```
def search_documents(query, top_k=3):
    query_vector = vectorizer.transform([query])
    scores = cosine_similarity(query_vector, document_vectors).flatten()
    order = scores.argsort()[::-1][:top_k]
    return [(int(i + 1), documents[i], float(scores[i])) for i in order]

for result in search_documents("Which method finds relevant documents?"):
    print(result)
```

Frontend / UI Implementation

Purpose: Documents the interface used to run searches and inspect results.

7.1 Framework Choice

A lightweight HTML, CSS and JavaScript interface is suitable for a small lab system. The user can view the document collection, enter a query and inspect the ranked output. The interface shown in the output illustration is a representative mock-up of the intended application; it is not a screenshot of a live deployed server.

7.2 Interface Features

- Editable document collection for quick experiments.
- Search box for a natural-language query.
- Ranked list of the top three documents.
- Similarity scores and bars to compare relevance.

Sample Input & Output

Purpose: Gives reproducible evidence of the system's expected behaviour.

Input Query	Processing Component	Retrieved Output
Which method finds relevant documents?	TF-IDF + cosine similarity	Information retrieval finds relevant documents for a user query. (highest score)
What represents term importance?	TF-IDF + cosine similarity	TF-IDF represents documents using term importance weights.
What identifies names and places?	TF-IDF + cosine similarity	Named Entity Recognition identifies names, places and organisations.

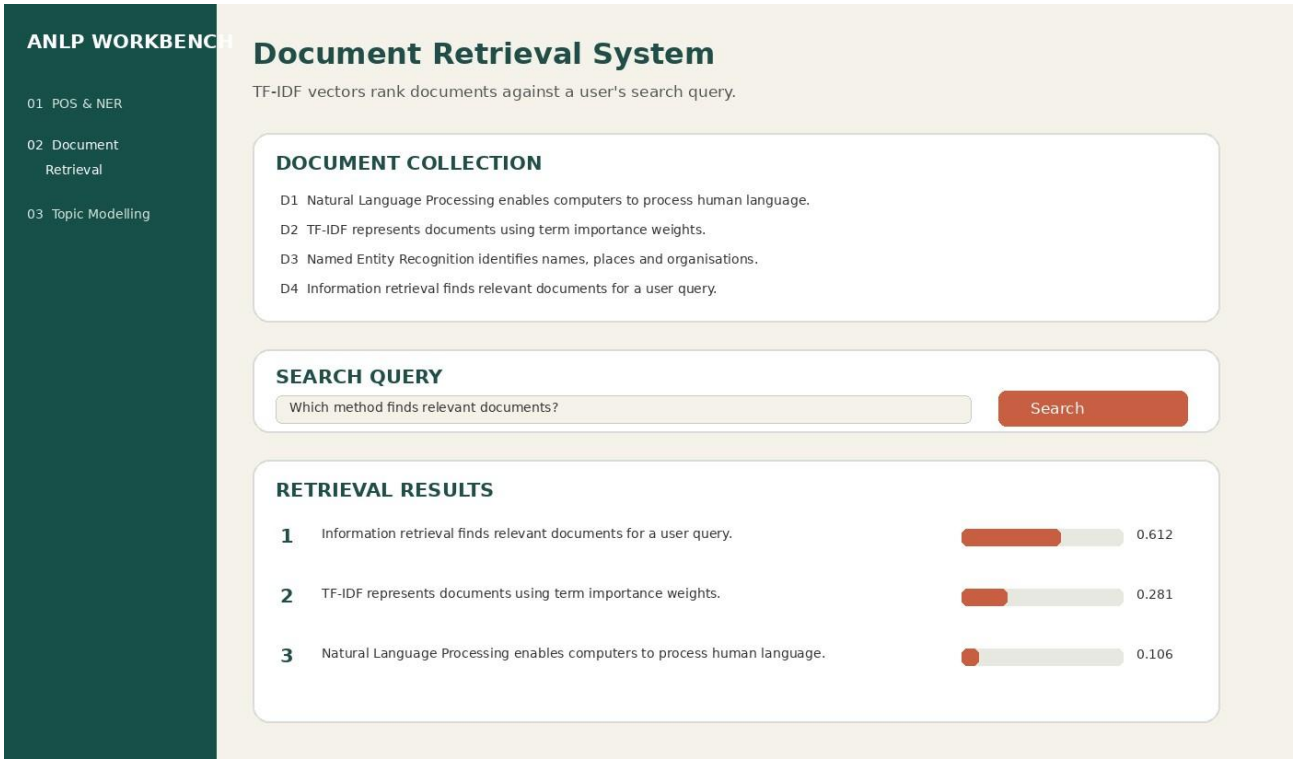


Figure 1. Document retrieval interface mock-up showing a query and ranked results.

Results (Deliverable-Specific Outputs)

Purpose: Presents the retrieval results for sample queries.

Retrieved Answers (sample queries)

- **Query 1:** “Which method finds relevant documents?” — the information retrieval document ranks first.
- **Query 2:** “What represents term importance?” — the TF-IDF document ranks first.
- **Query 3:** “What identifies names and places?” — the NER document ranks first.
- **Query 4:** “What processes human language?” — the NLP document ranks first.

Evaluation / Analysis

Purpose: Reviews whether the system ranks relevant documents for the test queries.

Performance Evaluation

Sub-task	Observed Result	Remarks
Top-ranked result	Relevant document expected for each sample query	The sample corpus contains direct keyword matches.
Score ranking	Documents sorted by cosine score	Scores support comparison between candidates.
Synonym handling	Limited	TF-IDF may miss related terms with different wording.
Corpus size	4 sample documents	A larger corpus is required for realistic evaluation.

Interpretation

- TF-IDF is simple and fast for small document collections.
- Cosine similarity ranks documents based on the terms shared with the query.
- The approach does not fully understand meaning, context or synonyms.
- Adding more documents and using sentence embeddings could improve retrieval quality.

Challenges Faced & Solutions

Challenge 1: Different document lengths

- **Problem:** Longer documents may contain more terms and distract from the main topic.
- **Solution:** TF-IDF normalisation and cosine similarity help compare vector direction rather than raw length.

Challenge 2: Queries with little term overlap

- **Problem:** A relevant document can receive a low score when the query uses different wording.
- **Solution:** Add synonyms or adopt a semantic embedding model for future improvement.

Challenge 3: Small corpus

- **Problem:** A small dataset cannot represent all possible search topics.
- **Solution:** Add a wider range of documents and evaluate with more queries.

Conclusion

An information-retrieval-based document search system was described using TF-IDF vectorisation and cosine similarity. The system transforms documents and a user query into vectors, scores each document and displays the highest-ranked results. The method is straightforward and useful for a small corpus, but its performance depends on word overlap. Semantic embeddings and a larger test collection are possible future extensions.

Project Repository (GitHub Link)

A repository can be created to store the Python code, sample corpus, interface files and README instructions. No repository URL is claimed here because a new repository has not been created for this report.

GitHub Repository URL	https://github.com/vijwalsheshadri/Vijwal1GA23AI057-ANLP
Repository Visibility	Public
Branch / Commit Used for Submission	main

References

- scikit-learn Documentation — Feature extraction: https://scikit-learn.org/stable/modules/feature_extraction.html
- scikit-learn Documentation — Pairwise metrics and cosine similarity: https://scikit-learn.org/stable/modules/metrics.html
- Manning, C. D., Raghavan, P., & Schütze, H. Introduction to Information Retrieval. Cambridge University Press.
- NLTK Documentation — Natural Language Toolkit: https://www.nltk.org/

ADVANCE NLP (AML23702)

LAB EXERCISE EVALUATION SHEET

Student Name	Vijwal Sheshadri
USN	1GA23AI057
Lab Exercise No.	2
Experiment Title	Information Retrieval-based Document Search System

Marks Awarded

Sl. No.	Evaluation Criteria	Max Marks	Marks Obtained
1	Problem Understanding & Scope	5	
2	Pipeline Design & Methodology	10	
3	Implementation Correctness	10	
4	Use of NLP Tools / Models	5	
5	Output Quality / Task Performance	10	
6	Analysis & Interpretation	5	
7	Documentation & Deliverables	2.5	
8	Presentation & Reproducibility	2.5	
	Total	**50**	

Overall Comments

Date of Submission	*Signature of Faculty*